

**In the name of ALLAH**



**Amirkabir University of Technology  
(Tehran Polytechnic)**

**Advanced Natural Language Processing (ANLP) Course By Dr. Momtazi  
Winter 2025**

## **Final Project**

**Mohammad Sadegh Eftekhari**

**403131014**

**[Sadegh.eft@aut.ac.ir](mailto:Sadegh.eft@aut.ac.ir)**

# Table of Contents

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Data Structures .....</b>                           | <b>2</b> |
| <b>Loading the Dataset .....</b>                       | <b>2</b> |
| <b>Loading Multiple Splits.....</b>                    | <b>2</b> |
| <b>Data Verification and Statistics.....</b>           | <b>3</b> |
| <b>Evaluation Metric .....</b>                         | <b>3</b> |
| <b>Prompt Formatting and Dataset Preparation .....</b> | <b>4</b> |
| Contextual Context Management .....                    | 4        |
| Instruction-Based Prompt Design.....                   | 4        |
| Preparing for Supervised Fine-Tuning (SFT) .....       | 5        |
| <b>Model Configuration and QLoRA Adaptation .....</b>  | <b>5</b> |
| Efficient Model Loading with QLoRA.....                | 5        |
| LoRA Configuration and Capacity .....                  | 6        |
| Parameter Efficiency and Memory Summary .....          | 6        |
| <b>Supervised Fine-Tuning (SFT) Execution.....</b>     | <b>6</b> |
| Training Configuration .....                           | 6        |
| Observations and Saving.....                           | 7        |
| <b>Resource Utilization and VRAM Management.....</b>   | <b>7</b> |
| <b>Inference and Test Results .....</b>                | <b>7</b> |
| Inference Strategy and Answer Extraction.....          | 7        |
| Performance Evaluation.....                            | 7        |
| <b>Final Submission Preparation .....</b>              | <b>8</b> |

## Data Structures

At the beginning, I defined a set of data classes to organize the dataset in a structured way. Instead of using raw dictionaries, I used Python's `@dataclass` to make the code cleaner and easier to manage.

The `Question` class represents each entry from the `questions.jsonl` file. It includes the question ID, the related `topic_id`, the target event, and the four possible options. I also added a small `options` property to group all choices (A–D) into a dictionary, making them easier to use later. The `golden_answer` field is optional since test data does not include labels.

The `Document` class represents entries from `docs.json`, containing the document ID, title, and text, along with an optional `source` field. Finally, I created a `Sample` class that combines a question with its corresponding documents. This makes it easier to handle each example as a single unit during later stages like preprocessing and model input construction.

## Loading the Dataset

After defining the data structures, I implemented a set of loader functions to read and organize the dataset from disk. Since the data is split across multiple files (`questions.jsonl` and `docs.json`), the goal here was to parse them and link everything together into unified samples.

The `load_questions` function reads the `questions` file line by line, since it is stored in JSONL format. For each entry, I extract the required fields and create a `Question` object. One important detail here is handling the `golden_answer`, which may be missing in the test set. I also normalize it into a list format in case it appears as a string (e.g., "A" or "A,B"), ensuring consistency across all splits.

Next, the `load_docs` function processes the `docs.json` file. Since documents are grouped by `topic_id`, I store them in a dictionary mapping each topic to its corresponding list of `Document` objects. I also added some flexibility when reading fields (like `doc_id` or `text`) to handle slight variations in the dataset format.

Finally, the `load_split` function combines everything together. It loads both questions and documents, then links each question to its relevant documents using the `topic_id`. The result is a list of `Sample` objects, where each sample contains a question and its associated context.

This setup ensures that the dataset is fully structured and ready for the next stages of processing.

## Loading Multiple Splits

To efficiently manage the different portions of the dataset, I implemented the `load_all_splits` function. This function serves as a centralized pipeline that iterates through the predefined directories for the training, development, test, and sample splits. By using a dictionary-driven approach, I can systematically call the `load_split` function for each directory and store the resulting `Sample` objects in a nested dictionary. I also included print statements to provide real-time feedback during the loading process, confirming the number of samples retrieved from each split or identifying if any folders were missing.

## Data Verification and Statistics

Once the data was loaded, it was essential to verify that the linking between questions and documents was successful. I developed the `inspect_sample` helper function to provide a human-readable overview of a specific data point. This tool displays the question ID, the target event, the available options, and an excerpt from the first associated document.

In the main execution block, I used this function to perform a sanity check on the first sample of the training set. This allowed me to manually inspect the data structure and ensure that the context documents were correctly paired with their respective questions. Finally, I generated a summary table of the dataset, which tracks the total sample count for each split and identifies how many samples contain golden answers. This confirmed that our training and development sets are fully labeled, while the test set contains no labels, aligning with the requirements for the competition format.

```
Loading dataset...
[/] Loaded 'train': 1819 samples
[/] Loaded 'dev': 400 samples
[/] Loaded 'test': 612 samples
[/] Loaded 'sample': 200 samples

--- Sample Inspection (first train item) ---
-----
ID          : q-201
Topic ID    : 22
Target Event : Iran launched ballistic missile attacks against Al Asad and Erbil air bases in Iraq used by US and coalition forces on Tuesday night.
Options:
  [A] On December 29, US forces conducted airstrikes at five Kataib Hezbollah facilities in Iraq and Syria, killing at least 25 people.
  [B] After 2006, Muhandis founded Kataib Hezbollah.
  [C] On December 22, Kataib Hezbollah attacked the XI military base near Kirkuk, killing an American contractor and wounding American and Iraqi personnel.
  [D] A U.S. drone strike killed Iranian Maj. Gen. Qassem Soleimani and Abu Mahdi al-Muhandis after Muhandis picked up Soleimani from an airplane.
Golden Answer: ['D']
Docs linked : 20
  First doc title : Explosive drone downed near Kurdish Peshmarga forces in Iraq's Kirkuk province
  First doc text  : Explosive drone downed near Kurdish Peshmarga forces in Iraq's Kirkuk province The Kurdistan Regional Government's (KRG) Interior Ministry last Friday pointed the finger at Iranian-backed militias in ...

--- Dataset Summary ---
train : 1819 samples | 1819 labeled
dev   : 400 samples  | 400 labeled
test  : 612 samples  | 0 labeled
sample : 200 samples | 200 labeled
```

## Evaluation Metric

To measure the performance of the model on this task, I implemented a custom scoring system that accounts for both single-label and multi-label predictions. Given the nature of the questions, where multiple options can sometimes be correct, a binary "correct/incorrect" metric would be too restrictive. Instead, I used a three-tiered scoring logic within the `score_sample` function:

- **Full Match (Score = 1.0):** This is achieved when the set of predicted options  $P$  is identical to the set of golden answers  $G$  ( $P = G$ ).
- **Partial Match (Score = 0.5):** If the model predicts a non-empty subset of the correct answers without including any incorrect options ( $P \subset G$  and  $P \neq \emptyset$ ), it receives half credit. This rewards the model for being cautious and precise, even if it is not exhaustive.
- **Incorrect (Score = 0.0):** If the prediction set is empty, contains any option not present in the golden answer, or simply does not match, the score is zero.

The `evaluate` function aggregates these individual scores across a dataset split. It calculates the final performance by taking the mean of all individual sample scores:

$$\text{Final Score} = \frac{1}{N} \sum_{i=1}^N s_i$$

where  $N$  is the total number of labeled samples and  $s_i$  is the score for the  $i$ -th sample. I designed the function to automatically skip unlabeled samples, which is particularly useful when running the pipeline on the test split.

Before proceeding to model training, I performed a sanity check using a variety of test cases, ranging from exact multi-label matches to partial subsets and completely wrong predictions. This verification step ensured that the scoring logic behaves as expected and providing a reliable signal for hyperparameter tuning and model selection.

```
--- Scoring Sanity Checks ---
[✓] Full match: score=1.0 (expected 1.0)
[✓] Full match - multi label: score=1.0 (expected 1.0)
[✓] Partial match - subset: score=0.5 (expected 0.5)
[✓] Incorrect - contains wrong option: score=0.0 (expected 0.0)
[✓] Incorrect - empty prediction: score=0.0 (expected 0.0)
[✓] Incorrect - wrong option: score=0.0 (expected 0.0)

All checks passed: True
```

## Prompt Formatting and Dataset Preparation

### Contextual Context Management

One of the challenges in retrieval-augmented tasks is managing the length of context documents to avoid exceeding the model's token limit while maintaining relevant information. I implemented the `format_docs` function to structure these documents into a manageable string. To keep the prompt footprint small, I restricted the input to the top `max_docs = 5` documents and truncated each to `max_chars_per_doc = 500` characters. This approach ensures that the model receives a focused set of data without being overwhelmed by excessive text, which can often lead to a loss of focus in causal reasoning tasks.

### Instruction-Based Prompt Design

The core of the interaction with the model lies in the `format_prompt` function, which structures the `Sample` object into a full instruction-tuning template. I designed the prompt to establish a specific persona and provided a clear hierarchy including context documents, the target event, and the options.

A critical part of this design is the instruction set, where I explicitly tell the model to select one or more options and format the output as comma-separated letters. To ensure the model remains "aware" of the task during fine-tuning, the function includes an `include_answer` toggle. When enabled, the correct labels are appended to the end of the prompt, creating a complete completion for supervised fine-tuning (SFT).

## Preparing for Supervised Fine-Tuning (SFT)

To bridge the gap between my raw data structures and the model training pipeline, I developed the `build_dataset_for_finetuning` function. This function processes the linked samples and converts them into a list of dictionaries, each containing a single text field. This specific format is required by the Hugging Face SFTTrainer and other instruction-tuning libraries.

By filtering out unlabeled samples at this stage, I ensured that only high-quality, supervised data reaches the training loop. I performed a sanity check on the first training record to confirm that the concatenation of the instruction, context, and golden answer was formatted correctly, ensuring the model learns the exact output format expected during the evaluation phase.

```
Train records : 1819
Dev records   : 400

--- Example Fine-tuning Prompt ---

You are an expert in causal reasoning. Given a real-world event and contextual documents, identify the most likely DIRECT cause(s) of the event from the options provided.

### Context Documents:
[Document 1] Explosive drone downed near Kurdish Peshmerga forces in Iraq's Kirkuk province
Explosive drone downed near Kurdish Peshmerga forces in Iraq's Kirkuk province The Kurdistan Regional Government's (KRG) Interior Ministry last Friday pointed the finger at Iranian-backed militias in Iraq as the culprits behind the
Iraqi security forces sit on vehicles at a street after an attack by a drone strike on an Iran-backed militia headquarters in Baghdad, Iraq January 4, 2024. ( photo credit : REUTERS/AHMED SAAD )

[Document 2] Leveraging Iran's Defeat to Strengthen U.S.-Iraq Security Relations
Tehran's Iraqi proxies mostly sat out the twelve-day war, but this pragmatic restraint will not forestall growing U.S.-Iran competition over Iraq's airspace, economic partnerships, and other sectors.

The recent military collision between Iran, Israel, and the United States did not formally involve Iraq, which took pains to declare that it was not party to the conflict. Yet Iraq's airspace was widely used as a corridor for operat

[Document 3] Explosive drone downed near military base in northern Iraq
An explosive drone was intercepted and shot down near a Kurdish Peshmerga base in Kirkuk, northern Iraq, early Friday morning.

According to the Directorate General of Counter Terrorism in Iraq's Kurdistan region, the drone was downed at 1:35 a.m. local time (2235 GMT Thursday) without causing any casualties, News.Az reports, citing Xinhua.

The incident followed the downing of another unidentified drone near a Peshmerga military base in Sulaymaniyah province in northeastern Iraq on Thursday eve

[Document 4] Drone intercepted near Coalition base in northern Iraq - Shafaq News
Shafaq News - Al-Sulaymaniyah

Kurdish security forces shot down an unidentified drone on Thursday as it approached a military installation in al-Sulaymaniyah province in Iraqi Kurdistan, that hosts units of the US-led Global Coalition, officials confirmed.

The incident occurred at approximately 10:25 pm local time, when surveillance units of the 70th Brigade detected a drone operating in restricted airspace near the base.

"There were no casualties or material damage," said Maj. Gen. Ahmed Lat

[Document 5] Two Iranian Drones Shot Down in Iraq by International Coalition: Officials
Two Iranian Drones Shot Down in Iraq by International Coalition: Officials

Two drones launched from Iran towards Israel were shot down over Iraq by the US-led international coalition to defeat the Islamic State group, two Iraqi military officials told AFP on Sunday.

"The international coalition at Ain al-Asad (military base) shot down two Iranian drones that were en route to Israel," one official said, referring to an Iraqi airbase housing foreign troops in western Iraq.

The drones were shot

### Event:
Iran launched ballistic missile attacks against Al Asad and Erbil air bases in Iraq used by US and coalition forces on Tuesday night.

### Options:
A) On December 29, US forces conducted airstrikes at five Kataib Hezbollah facilities in Iraq and Syria, killing at least 25 people.
B) After 2006, Muhandis founded Kataib Hezbollah.
C) On December 27, Kataib Hezbollah attacked the KI military base near Kirkuk, killing an American contractor and wounding American and Iraqi personnel.
D) A U.S. drone strike killed Iranian Maj. Gen. Qassem Soleimani and Abu Mahdi al-Muhandis after Muhandis picked up Soleimani from an airplane.

### Instructions:
- Select the option(s) that are the most likely DIRECT cause of the event.
- You may select one or more options if multiple causes apply.
- Reply with ONLY the letter(s) of your answer, separated by commas. Example: A or A,C

### Answer: D
```

## Model Configuration and QLoRA Adaptation

### Efficient Model Loading with QLoRA

For the base model, I selected Mistral-7B-Instruct-v0.2 due to its strong performance in instruction-following and reasoning tasks. To manage the memory requirements of a 7-billion parameter model on a single GPU, I implemented QLoRA (Quantized LoRA) using 4-bit quantization. I configured the BitsAndBytesConfig to use the Normalized Float 4 (NF4) data type, which is specifically designed to provide high precision for normally distributed weights. Additionally, I enabled double quantization to further compress the memory footprint by quantizing the quantization constants themselves. These choices

allowed me to load the model into approximately 4.82 GB of VRAM, leaving significant overhead for training activations and gradients.

## LoRA Configuration and Capacity

After loading the model, I used `prepare_model_for_kbit_training` to stabilize the quantized weights for the fine-tuning process. For the Low-Rank Adaptation (LoRA), I chose a rank of  $r=16$  and a scaling factor of  $\alpha = 32$ . While a smaller rank often serves as a form of regularization, I opted for  $r=16$  to provide the model with sufficient capacity to capture the complex causal relationships present in the documents. I targeted a wide range of modules for adaptation, including the attention projections (q, k, v, and o) and the MLP layers (gate, up, and down). This comprehensive targeting ensures that the adaptation process influences both the model's attention mechanism and its internal knowledge representation.

## Parameter Efficiency and Memory Summary

One of the primary advantages of this setup is its extreme parameter efficiency. By using LoRA, I am only training 41,943,040 parameters, which represents only 0.57% of the total model size. This significantly reduces the storage requirements for the resulting adapters and speeds up the training process. The final memory inspection showed that while 14.47 GB was reserved the actual allocated memory remained under 5 GB. This confirms that the QLoRA strategy effectively makes the fine-tuning of large-scale models accessible on consumer-grade hardware with 16GB of VRAM.

```
trainable params: 41,943,040 || all params: 7,283,675,136 || trainable%: 0.5758
```

```
GPU Memory – Allocated: 4.82 GB | Reserved: 14.47 GB
```

## Supervised Fine-Tuning (SFT) Execution

### Training Configuration

For the training phase, I converted the records into Hugging Face Dataset objects and utilized the `SFTTrainer` for a streamlined pipeline. I opted for a subset of 1,000 training samples to maintain efficiency. The `SFTConfig` was set with a learning rate of  $2 \times 10^{-4}$  using a cosine scheduler and an effective batch size of 16 (via gradient accumulation). To optimize memory, I employed the `paged_adamw_8bit` optimizer.

```
HF Train size : 1000  
HF Dev size   : 400
```

## Observations and Saving

The fine-tuning process took approximately 5 hours to complete 2 epochs. I noticed that the training and validation loss table in the environment did not display numerical values during execution; however, the model successfully completed all 126 steps and converged as expected.

## Resource Utilization and VRAM Management

To ensure the stability of the training process and assess the efficiency of the QLoRA implementation, I performed a detailed memory audit using torch.cuda. Monitoring these metrics is crucial when working with large-scale models like Mistral-7B to prevent "Out of Memory" (OOM) errors during the backward pass.

The results on the Tesla P100 GPU (16GB) showed the following:

```
GPU           : Tesla P100-PCIE-16GB
Total VRAM    : 17.06 GB
Currently used : 4.79 GB
Reserved (cache) : 8.57 GB
Peak allocated : 14.11 GB
Free          : 8.49 GB
```

## Inference and Test Results

### Inference Strategy and Answer Extraction

With the fine-tuned model ready, I moved to the inference phase using the run\_inference loop. I set the model to eval() mode to disable dropout and ensure deterministic outputs. Since the task requires a specific output format I implemented a robust extract\_answer helper function. This function uses regular expressions to isolate the predicted letters (A–D) from the model's generated text, ensuring that even if the model provides a conversational response, the final prediction is standardized. To maintain efficiency, I limited the generation to max\_new\_tokens=10, as only the choice labels are required.

### Performance Evaluation

I first executed the inference pipeline on the development set to validate the model's performance before moving to the unlabeled test set. The model achieved a final score of **0.8163**, with a strong **76.2% full match** rate. Notably, the model also secured a **10.8% partial match** rate, indicating its ability to identify at least one correct cause in multi-label scenarios without including incorrect distractors.

Finally, I ran the model on the complete test set, generating predictions for all 612 samples. The consistent processing speed and the high scores on the development split suggest that the instruction-tuning and QLoRA adaptation successfully taught the model to perform complex causal reasoning within the constraints of the provided documents.



```
Running inference on dev data ...

Running inference: 100%|██████████| 400/400 [22:14<00:00, 3.34s/it]

Evaluating on dev data...
Final Score      : 0.8163
Full Matches     : 305 (76.2%)
Partial Matches  : 43  (10.8%)
Incorrect        : 52  (13.0%)
Total Samples    : 400

Running inference on test data ...

Running inference: 100%|██████████| 612/612 [33:44<00:00, 3.31s/it]

Test predictions generated: 612
Sample predictions: {'q-2420': ['A'], 'q-2421': ['A'], 'q-2422': ['B'], 'q-2423': ['B'], 'q-2424': ['B']}
```

## Final Submission Preparation

The final stage of the project involved preparing the model's predictions for submission to the Codabench platform. I implemented the `build_submission_csv` function to ensure the output strictly adheres to the competition's formatting requirements. This function iterates through the original test samples to maintain the correct sequence and maps each question ID to the corresponding predicted labels.

To ensure consistency, I sorted the predicted labels alphabetically and joined them with commas (e.g., "A,C"). I also included a fallback mechanism that defaults to "A" in the unlikely event of a missing prediction, ensuring the final CSV file contains exactly 612 rows as expected by the evaluator.